



Ciência da Computação
Estrutura de Dados

A Linguagem C

Aldo Moura
aldo.moura@souunit.com.br

Roteiro



- Expressões aritméticas
- Expressões relacionais e lógica
- Estrutura decisão
- Estrutura de repetição
- Contador
- Acumulador
- Quebra de laço
- Visibilidade de variáveis
- Conversão de tipo

Expressões

Sentenças geradas a partir da combinação de variáveis e constantes para processar determinadas ações.

A maioria dos operadores são binários. Requerem dois operandos (dados) para o processamento.

```
<operando_1> <OPERADOR> <operando_2>
```

Dependendo do tipo de operador utilizado, a expressão pode ser classificada como:

- Aritmética: **num1 + 2**
- Relacional: **num1 >= 10**
- Lógica: **num1 >= 0 && num1 <= 10**

Expressões Aritméticas

Utilizados para operações aritméticas entre operandos do tipo numéricos (int, double, float, ...).

Operadores aritméticos:

+	Adição
-	**Subtração**
*	**Multiplicação**
/	**Divisão**
%	**Resto de divisão**

Exemplos:

```
100+200 // resulta 300
300-50  // resulta 250
40*3    // resulta 120
5/2     // resulta 2, a divisão entre inteiros resulta em
        // um quociente inteiro. A parte decimal é truncada
4/7     // resulta 0  (0.571 onde o .571 é truncado)
5%2     // resulta 1
```

Expressões Aritméticas

As expressões podem ser simples (apenas um operador) ou composta (mais de um operador).

100 + 200 // expressão simples

10 + 2 + 300 // expressão composta

Prioridades dos operadores aritméticos:

1º	**Multiplicação, divisão e resto de divisão**
2º	**Adição e subtração**

Pode-se usar parênteses para alterar a ordem de prioridade.

Exemplos:

```
10+20*30              // resulta 610
50-8/2                // resulta 46
(10+20)*30            // resulta 900
(50-8)/2              // resulta 21
40+10*(150-60)/3      // resulta 340
```

Expressões Relacionais

Utilizados para comparar os valores dos operandos.

O resultado de uma comparação pode ser 1 (comparação verdadeira) ou 0 (comparação falsa).

Operador Relacional	Expressão Relacional	Significado	Exemplo	Resultado
==	x == y	x é **igual** a y	10 == 10	1
!=	x != y	x é **diferente** de y	10 != 10	0
>	x > y	x é **maior que** y	150 > 200	0
<	x < y	x é **menor que** y	150 < 200	1
>=	x >= y	x é **maior ou igual que** y	300 >= 540	0
<=	x <= y	x é **menor ou igual que** y	40 <= 40	1

Expressões Lógicas

Permite a elaboração de expressões relacionais compostas.

Operador Lógico	Significado	Expressão Lógica	Significado
&&	E	(x>y) && (y>z)	Resulta em 1(verdadeiro) se e somente se ambas comparações forem 1(verdadeiro)
||	OU	(x>y) || (x>z)	Resulta em 1(verdadeiro)se ambas ou quaisquer uma das comparações forem 1(verdadeiro)
!	NÃO lógico	!(x > y)	Inverte o significado de uma comparação. • Se a condição for 1(verdadeiro) resulta 0(falso) • Se a condição for 0(falso) resulta 1(verdadeiro)

Expressões Lógicas

Exemplos:

```
(3<7) && (7<9)

(3>7) || (3<5)

!(23>18)

(5>8) && (7>5) || (4!=8)

!(4==4) && (6>3) || (3>=3) && (2<=5)
```

PRIORIDADE	OPERADOR
1º	!
2º	&&
3º	||

Tabela Verdade

Mostra as combinações de valores possíveis entre duas comparações envolvidas com operadores lógicos && e || e o valor resultante.

&& (E)		
Condição 1	Condição 2	Resultado
1	1	1
1	0	0
0	1	0
0	0	0

|| (OU)		
Condição 1	Condição 2	Resultado
1	1	1
1	0	1
0	1	1
0	0	0

Exemplos:

```
(3<7) && (7<9)
  1    &&   1     => 1

(3>7) || (3<5)
  0    ||   1     => 1

(5!=5) &&  (2==2)
   0    &&    1   => 0

(4>2) || (2>1)
  0    ||   0     => 0
```

Tabela Verdade ! (não lógico)

Mostra os valores possíveis de uma condição e o valor resultante desta condição com o operador lógico ! (NÃO lógico).

Exemplos:

!(23>18)

! (1) => 0

!(44<20)

! (0) => 1

! (NÃO lógico)	
Condição	Resultado
0	1
1	0

Estrutura de Decisão

- de Decisão
- de Desvio
- de Seleção
- Condicional

A partir do resultado de uma expressão lógica altera o fluxo de execução das instruções de um programa.

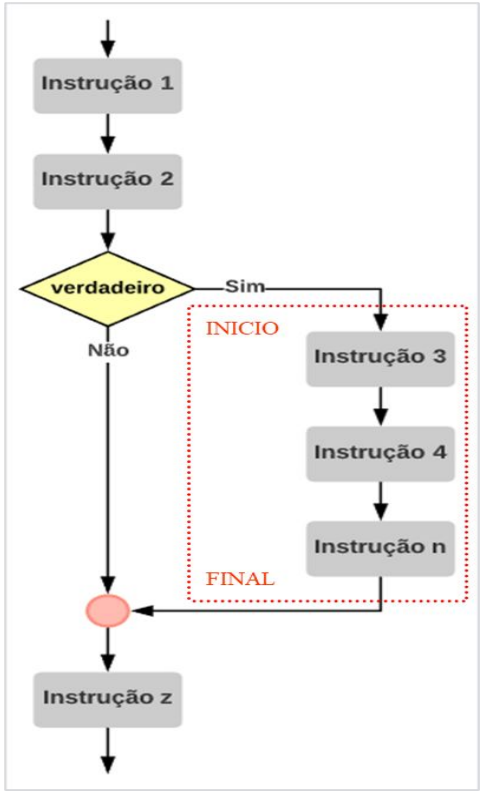
- Simples
- Composta
- Encadeada
- Múltipla

Estrutura de Decisão - Simples

Se o valor da expressão lógica for 1 (verdadeiro), será executado o bloco de instruções da estrutura.

Sintaxe C:

```
if <(expressão lógica)>{
   instruções;
}
```




```c
int idade;

printf("Idade: ");
scanf("%d", &idade);

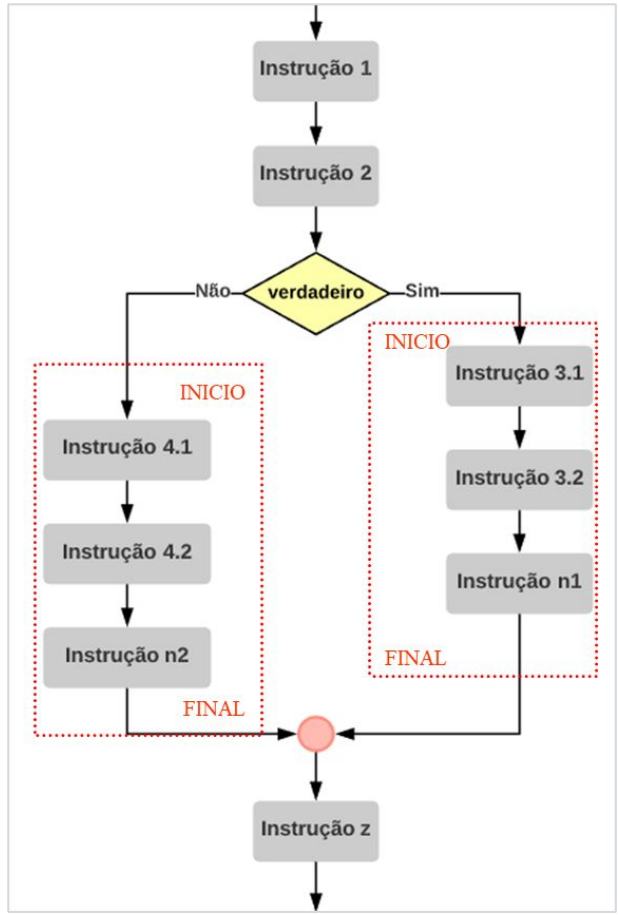
if (idade<16){
    printf("Não pode votar!");
}
```

Estrutura de Decisão - Composta

- Se o valor da expressão lógica for 1 (verdadeiro) será executado o bloco de instruções SIM da estrutura.
- Se o valor da expressão lógica for 0 (falso) executa o bloco de instruções NÃO da estrutura.

Sintaxe C:

```
if <(expressão lógica)>{
   instruções_bloco_SIM;
} else {
   instruções_bloco_NÃO;
}
```



Estrutura de Decisão - Composta

```c
float nota1, nota2, media;

printf("1ª nota: ");
scanf("%f", &nota1);
printf("2ª nota: ");
scanf("%f", &nota2);

media = (nota1 + nota2) / 2;
printf("Média: %.1f", media);

if (media >= 7) {
    printf("\nAprovado");
} else {
    printf("\nReprovado");
}
```

Estrutura de Decisão - Encadeada

As instruções de um bloco pode conter outras estruturas de decisão, tantas quantos forem necessárias para resolver o problema proposto.

```c
int numero1, numero2;

printf("1º número: ");
scanf("%d", &numero1);
printf("2º número: ");
scanf("%d", &numero2);

if (numero1 > numero2) {
    printf("O 1º número é maior");
} else {
    if (numero2 > numero1) {
        printf("O 2º número é maior");
    } else {
        printf("Os números são iguais");
    }
}
```

```c
int numero1, numero2;

printf("1º número: ");
scanf("%d", &numero1);
printf("2º número: ");
scanf("%d", &numero2);

if (numero1 > numero2) {
    printf("O 1º número é maior");
} else if (numero2 > numero1) {
    printf("O 2º número é maior");
} else {
    printf("Os números são iguais");
}
```

Estrutura de Decisão Encadeada

```c
#include <stdio.h>

int main()
{
    char tipoCurso;

    printf("Tipo do curso: [G/T]? ");
    scanf(" %c", &tipoCurso);

    if (tipoCurso == 'G' || tipoCurso == 'g') {
        printf("\nGraduação");
    } else if (tipoCurso == 'T' || tipoCurso == 't') {
        printf("\nTécnico");
    } else {
        printf("\nOpção inválida");
    }

    return 0;
}
```

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    char tipoCurso;

    printf("Tipo do curso: [G/T]? ");
    scanf(" %c", &tipoCurso);

    tipoCurso = toupper(tipoCurso);

    if (tipoCurso == 'G') {
        printf("\nGraduação");
    } else if (tipoCurso == 'T') {
        printf("\nTécnico");
    } else {
        printf("\nOpção inválida");
    }

    return 0;
}
```

Estrutura de Decisão Múltipla

Define as instruções a serem executadas com base na comparação de uma expressão com uma sequência de valores especificados nas cláusulas cases.

- Se uma das comparações for **1** (verdadeira), executa os instruções seguintes até que o final da estrutura ou até um comando break (encerra a estrutura).

- Se todas as comparações forem 0 (falsas), executa, se houver, a cláusula default.

Sintaxe C:

```
switch <(expressão inteira)>{
    case <valor_1:>
       conjunto_instruções_1;
       [break;]
    case <valor_2:>
       conjunto_instruções_2;
       [break;]
    ...
    case <valor_n:>
       conjunto_instruções_n;
       [break;]
    <default:
       conjunto_instruções_n;>
}
```

Estrutura de Decisão Múltipla – Exemplo 1

```c
#include <stdio.h>

int main()
{
    char operador;
    int num1, num2, resposta;

    printf("1º número: " );
    scanf("%d", &num1);
    printf("2º número: " );
    scanf("%d", &num2);

    printf("Operador [+/-]?");
    scanf(" %c", &operador);

    switch (operador) {
        case '+':
            resposta=num1+num2;
            printf("Adição: %d + %d = %d", num1, num2, resposta);
            break;
        case '-':
            resposta=num1-num2;
            printf("Subtração: %d - %d = %d", num1, num2, resposta);
            break;
        default:
            printf("Opção inválida");
    }
    return 0;
}
```

Estrutura de Decisão Múltipla – Exemplo 2

```c
#include <stdio.h>
#define PERC_JUROS_MES 1.35

int main()
{
    float valorInicial, valorFinal, juros = 0;
    int meses;

    printf("Valor inicial da dívida: " );
    scanf("%f", &valorInicial);

    printf("Meses de atraso [1-5]: " );
    scanf("%d", &meses);

    switch (meses) {
        case 5:
            juros = juros + PERC_JUROS_MES;
        case 4:
            juros = juros + PERC_JUROS_MES;
        case 3:
            juros = juros + PERC_JUROS_MES;
        case 2:
            juros = juros + PERC_JUROS_MES;
        case 1:
            juros = juros + PERC_JUROS_MES;
            valorFinal = (1 + juros / 100) * valorInicial;
            printf("\nPercentual de juros  : %f %", juros);
            printf("\nValor final da dívida: R$ %.2f", valorFinal);
           break;
        default:
           printf("Número de meses inválido");
    }
    return 0;
}
```

Estrutura de Decisão Múltipla – Exemplo 3

```c
int meses;

printf("Número do meses [1-12]: " );
scanf("%d", &meses);

switch (meses) {
    case 1:
    case 2:
    case 3:
    case 4:
    case 5:
    case 6:
        printf("Primeiro semestre");
        break;
    case 7:
    case 8:
    case 9:
    case 10:
    case 11:
    case 12:
        printf("Segundo semestre");
        break;
    default:
       printf("Número do meses inválido");
}
```

Estrutura de Repetição

- Iteração
- Laço (loop)

Permite a repetição de um bloco de instruções enquanto alguma condição esteja sendo satisfeita.



- do ... while
- while
- for

Estrutura de Repetição – do ... while

Repete um bloco de instruções enquanto uma condição resultar em true.

A condição é verificada na base do bloco, ocasionando a iteração pelo menos uma vez.

Sintaxe:

```
do {
    instrução_1
    instrução_2
    ...
    instrução_n
} while (<condição>);
```

Estrutura de Repetição – do ... while

Escreva um programa para receber o tipo do curso (G/P).

Criticar para só aceitar o caractere G ou P.

Após ser recebido o tipo de curso correto, mostrar “Graduação” ou “Pós-graduação”, conforme a letra recebida.

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    char tipoCurso;

    do {
        printf("Tipo do curso: [G/T]? ");
        scanf(" %c", &tipoCurso);
        tipoCurso = toupper(tipoCurso);
        if (tipoCurso != 'G' && tipoCurso != 'P') {
            printf("\nTipo incorreto. Digite G ou T\n");
        }
    } while (tipoCurso != 'G' && tipoCurso != 'P');

    if (tipoCurso == 'G') {
        printf("\nGraduação");
    } else  {
        printf("\nTécnico");
    }

    return 0;
}
```

Contador

É uma variável que, a partir de um valor inicial, tem seu valor incrementado ou decrementado em um valor fixo, por atribuição, sempre que for executado.

Sintaxe: CONTADOR = CONTADOR <operador> 1;

Escreva um programa para receber o tipo do curso (G/P).

Criticar para só aceitar o caractere G ou P.

Após acertar informar em que tentativa o usuário acertou o tipo de curso.

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    int contador = 0;
    char tipoCurso;

    do {
        printf("Tipo do curso: [G/T]? ");
        scanf(" %c", &tipoCurso);
        tipoCurso = toupper(tipoCurso);
        if (tipoCurso != 'G' && tipoCurso != 'P') {
            printf("\nTipo incorreto. Digite G ou T\n");
        }

        contador = contador + 1;

    } while (tipoCurso != 'G' && tipoCurso != 'P');

    if (tipoCurso == 'G') {
        printf("\nGraduação");
    } else  {
        printf("\nTécnico");
    }
    printf("\nVocê acertou na %dª tentantiva", contador);

    return 0;
}
```

Acumulador

É uma variável que, a partir de um valor inicial, tem seu valor incrementado ou decrementado em um valor variável, por atribuição, sempre que for executado.

Sintaxe: ACUMULADOR = ACUMULADOR <operador> 1;

Escreva um programa para receber o preço de 4 produtos e no final mostrar o valor total dos produtos.

```c
#include <stdio.h>

int main()
{
    int qtdeProdutos = 4;
    float valorTotal = 0;
    float preco;

    do {
        printf("Preço R$: ");
        scanf("%f", &preco);
        valorTotal = valorTotal + preco;
        qtdeProdutos = qtdeProdutos - 1;
    } while (qtdeProdutos > 0);

    printf("\nValor total: R$ %.2f", valorTotal);

    return 0;
}
```

Estrutura de Repetição - while

Repete um bloco de instruções enquanto uma condição resultar em true.

A condição é verificada no topo do bloco, podendo não ocorrer nenhuma iteração caso a condição resulte em false na primeira verificação.

Sintaxe:

```
while (<condição>) {
    instrução_1
    instrução_2
    ...
    instrução_n
}
```

Estrutura de Repetição – while: exemplo

Escreva um programa para:

- Receber quantidade de aulas ministradas.
- Para cada aula ministrada receber a quantidade de alunos presentes.
- No final mostrar a quantidade total de alunos nas aulas.

```c
#include <stdio.h>

int main()
{
    int repeticao = 1;
    int totalAlunos = 0;
    int qtdeAulas;
    int alunosPorAula;

    printf("Quantidade de aulas ministradas: ");
    scanf("%d", &qtdeAulas);

    while (repeticao <=qtdeAulas) {
        printf("Quantidade de alunos na %dª aula: ", repeticao);
        scanf("%d", &alunosPorAula);

        totalAlunos = totalAlunos + alunosPorAula;
        repeticao = repeticao + 1;
    }

    printf("\nTotal de alunos: %d", totalAlunos);

    return 0;
}
```

Quebra de Laço

Break: Encerra a estrutura de repetição

Continue: Desviando o fluxo para a condição que controla a repetição

```c
#include <stdio.h>
#include <ctype.h>

int main()
{
    char caractere;

    do {
        printf("Digite uma letra: ");
        scanf(" %c", &caractere);
        if ( isalpha(caractere) == 0) {
            printf("\nNão é uma Letra!!!\n");
            continue;
        }
        break;
    } while (1); // Laço infinito

    printf("\nLetra digitada: %c", caractere);

    return 0;
}
```

Estrutura de Repetição - for

Repete um bloco de instruções uma determinada quantidade de vezes.

Sintaxe:

```
for (int var = vl_inicial; condição; incremento) {
    instrução_1;
    instrução_2;
    ...
    instrução_n;
}
```

Estrutura de Repetição – for: exemplo

Escreva um programa para receber o preço de 4 produtos e no final mostrar o valor total dos produtos.

```c
#include <stdio.h>

int main()
{

    float preco;
    float valorTotal = 0;

    for (int i = 1; i <= 4; i = i + 1) {

        printf("Preço do %dº produto R$: ", i);
        scanf("%f", &preco);
        valorTotal = valorTotal + preco;
    }

    printf("\nValor total dos produtos: R$ %.f", valorTotal);

    return 0;
}
```

Alcance ou Visibilidade de uma Variável

Dependendo de onde uma variável for declarada, ela será acessível (visível) de qualquer lugar do código, ou apenas em uma porção confinada do mesmo (dentro de uma função, por exemplo), isto é chamado de alcance ou visibilidade de uma variável.

As variáveis podem ser do tipo:

Local

Global

Variável Local

Uma variável é considerada de alcance local, quando ela é declarada dentro de um bloco de instruções (função, laço, estrutura condicional, etc). Ela terá um alcance limitado a este único bloco de instrução, ou seja, não pode ser usada em outro lugar.

```c
#include <stdio.h>
int main()
{
    int x=20, y=30 ;
    if (x>0)
    {
       int x=10;
       printf("Valor de x+y: %i\n", x+y);
    }
    printf("Valor de x+y: %i\n", x+y);
    return 0;
}
```

```c
#include <stdio.h>
void f1()
{
    int x = 5;
    printf("Valor de x: %i\n",x);
}

int main()
{
    int x=10;
    f1();
    printf("Valor de x: %i\n", x);
    return 0;
}
```

Variável Global

Uma variável é considerada de alcance global, quando ela é declarada fora de qualquer função. Ela terá um alcance global e poderá ser acessada em qualquer ponto do programa.

```c
#include <stdio.h>
int x = 10;

void f1()
{
    x++;
}

int main()
{
    f1();
    printf("Valor de x: %i\n",x);
    return 0;
}
```

Parâmetro de uma Função é uma Variável Local

```c
#include <stdio.h>
int f1(int y)
{
    return y * 2;
}

int main()
{
    int x=10;
    printf("Valor de x: %i", f1(x));
    return 0;
}
```

Variável Estática

Uma variável do tipo estática, tem como principal característica a conservação do seu valor durante a execução do programa.

```c
#include <stdio.h>
int f1()
{
    static int x = 0;
    x = x + 1;
    return x;
}

int main()
{
    for (int i=0; i<3; i++) {
        printf("Valor de x: %d\n", f1());
    }
    return 0;
}
```

Conversão de Tipos (cast)

A conversão de tipos de variáveis, também chamada de cast de variáveis, tem a função de alterar o tipo de dados armazenados por uma variável.

```c
#include <stdio.h>
int main()
{
    int x=10, y=3;
    float z = (float) x / y;
    printf("Valor de z: %.2f\n", z);
    return 0;
}
```